

```
word_list = [word.replace(' ', '') for word in word_list]
word_list = [word.replace('(', '') for word in word_list]
word_list.remove('')
```

These lines use the `remove()` and `replace()` functions to replace punctuation with empty strings and then remove the empty strings. Now that we have a suitable word list, we'll be able to recognize invalid words more accurately. We can rerun our check for invalid words using our new `word_list` and get better results:

```
between_spaces_notvalid = [loc for loc in between_spaces if \text[loc[0]:loc[1]]
not in word_list]
```

When we print the list `between_spaces_notvalid`, we get a shorter and more accurate list:

```
[(4, 16), (24, 30), (35, 45), (55, 68), (72, 78), (93, 105), (112, 119), (120,
123)]
```

Now that we have found the invalid potential words in our text, we'll check in our word list for words that could be combined to form those invalid words. We can begin by looking for words that start just after a space. These words could be the first half of an invalid word:

```
partial_words = [loc for loc in locs if loc[0] in spacestarts_affine and \loc[1]
not in spacestarts]
```

Our list comprehension iterates over every element of our `locs` variable, which contains the location of every word in the text. It checks whether `locs[0]`, the beginning of the word, is in `spacestarts_affine`, a list containing the characters that come just after a space. Then it checks whether `loc[1]` is not in `spacestarts`, which checks whether the word ends where a space begins. If a word starts after a space and doesn't end at the same place as a space, we put it in our `partial_words` variable, because this could be a word that needs to have a space inserted after it.

Next, let's look for words that end with a space. These could be the second half of an invalid word. To find them, we make some small changes to the previous logic: